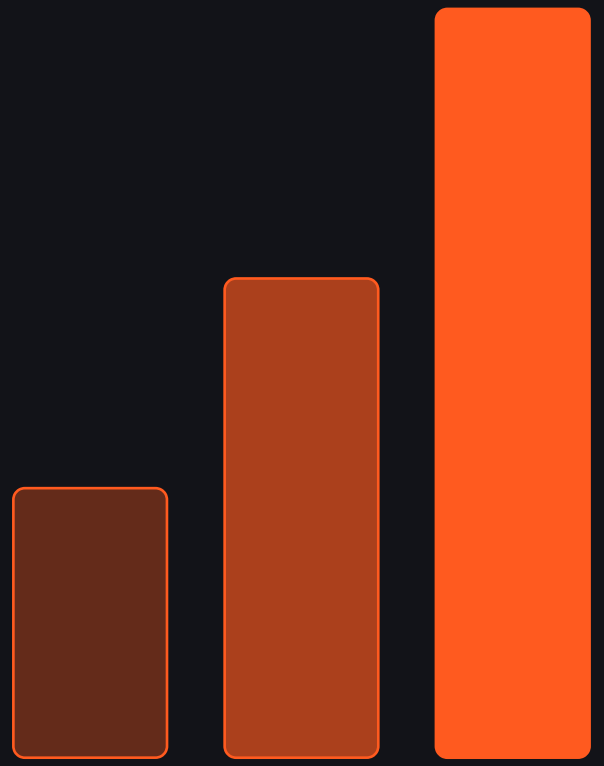
Abschlussprojekt Passerelle · BFH

GYM PLAN GENERATOR

Trainingspläne von der KI, geprüft und berechnet von meinem Java-Code.

Daniil Kryvoruchko · 29. Oktober 2026



DAS PROBLEM

Fertige Trainingspläne passen selten. Und ein Chatbot liefert Text, den niemand prüft.

Zu allgemein

Pläne aus dem Internet ignorieren Ziel, Zeit und Equipment.

Unkontrolliert

KI-Text kann falsche Satzzahlen oder unpassende Übungen enthalten.

Zu wenig Detail

Erfahrene Sportler brauchen RIR, Volumen pro Muskel und ein Kalorienziel.

ZIELE



1

Ein Plan, der passt

Abgestimmt auf Ziel, Erfahrung, Equipment, Trainingstage und Zeit pro Einheit.

2

Der KI nicht blind vertrauen

Ein Schema erzwingt die Form, mein Code prüft den Inhalt und rechnet die Zahlen selbst.

TECH-STACK





Backend

- Java
- Spring Boot 4
- Bean Validation
- Maven



Frontend

- Thymeleaf
- HTML und CSS
- JavaScript



KI und Tools

- Gemini API
- JSON-Schema
- IntelliJ
- Git und GitHub

SO FUNKTIONIERT ES



01

Formular

14 Eingaben, serverseitig geprüft

02

Prompt

PromptBuilder baut den Auftrag an Gemini

03

Prüfung

JSON einlesen, PlanChecker prüft den Inhalt

04

Ergebnis

Tabellen, Kalorien, Makros und Volumen

Ist der Plan ungültig, gibt es einen zweiten Versuch. Danach erscheint eine verständliche Fehlermeldung statt eines Absturzes.

LIVE-DEMO

Formular, Validierung, ein echter Plan, drei Sprachen, Kopieren und Drucken.

FUNKTIONEN

Validierung

Alle Eingaben werden geprüft, mit klaren Fehlermeldungen.

Plan als Tabellen

Pro Tag: Muskel, Sätze, Wiederholungen, RIR und Pause.

Volumen

Sätze pro Muskelgruppe und Woche, Abweichungen markiert.

Ernährung

Kalorien, Makros und erwarteter Gewichtstrend.

Drei Sprachen

Der ganze Plan auf Deutsch, Englisch oder Ukrainisch.

Teilen

Als Text kopieren, drucken oder als PDF speichern.

UMGESETZTE FEATURES



Feature	Umsetzung
Ernährungsziele	Cut, Aggressive Cut, Lean Bulk, Bulk
Körperfett optional	Dann Katch-McArdle statt Mifflin-St Jeor
Plan-Check	Tage, Sätze, RIR, Pausen; max. 2 Versuche
Fehlerbehandlung	Fehler 429 und 503 als verständliche Meldung
Mehrsprachigkeit	Message-Dateien, Sprache aus dem Formular

AUFBAU DES CODES

Web

Nimmt Anfragen an

`PlanController`
`PlanRequest`
`Thymeleaf-Seiten`

Logik

Steuert, prüft, rechnet

`PlanService`
`PlanChecker`
`NutritionCalculator`

KI

Spricht mit Gemini

`GeminiClient`
`PromptBuilder`
`GeminiException`

Spring erstellt die Klassen und verbindet sie per Dependency Injection über den Konstruktor.

CODE: PLAN ERZEUGEN

Antwort holen, einlesen, prüfen. Nur ein gültiger Plan wird angezeigt.

Maximal zwei Versuche: Das schont das kostenlose API-Kontingent.

Aus PlanService, vereinfacht

```
for (int i = 1; i <= 2; i++) {
    String text = geminiClient.generate(prompt);
    Plan plan = parse(text);
    List<String> problems =
        planChecker.check(plan, days);
    if (problems.isEmpty()) {
        return plan;
    }
}
throw new GeminiException(
    "Kein gültiger Plan");
```

CODE: KALORIEN

Die Zahlen rechnet mein Code, nicht die KI: nachvollziehbar und testbar.

≈ 2240 kcal

Beispiel: 80 kg, 180 cm, 25 Jahre, 4 Tage, Cut. 176 g Protein, 62 g Fett, 244 g Kohlenhydrate.

```
// Grundumsatz (Mifflin-St Jeor)
double bmr = 10 * weight
    + 6.25 * height - 5 * age
    + (male ? 5 : -161);
double maintenance =
    bmr * activityFactor(days);
double target = maintenance
    * (1 + calorieChange(goal));
// Schutz: Mindestkalorien
target = Math.max(target, floor);
```

PROBLEME UND LÖSUNGEN



Gemini überlastet (503)	➡	Auf Flash-Lite gewechselt, Fehler als verständliche Meldung
KI-Antworten fehlerhaft	➡	JSON-Schema plus PlanChecker, sonst zweiter Versuch
API-Schlüssel schützen	➡	Nur als Umgebungsvariable, nie im Code oder auf GitHub
Spring Boot war neu	➡	Schritt für Schritt in kleinen Lektionen gelernt

AUSBLICK

Benutzerkonten

Pläne speichern und wieder öffnen

Trainingsprotokoll

Gewichte eintragen, Fortschritt sehen

Online stellen

Mit Tageslimits pro Nutzer

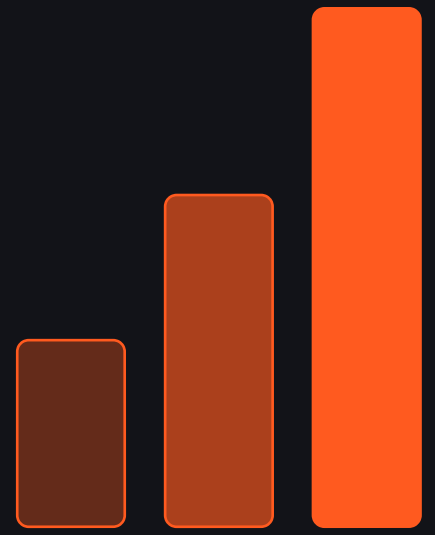
Mehr Optionen

Einheiten lb und Zoll, mehr Tests

Fazit

DIE KI LIEFERT IDEEN. MEIN CODE SORGT FÜR STRUKTUR, PRÜFUNG UND ZAHLEN.

Gelernt: Spring Boot, Validierung, Dependency Injection und der sichere Umgang mit einer KI-Schnittstelle.



DANKE

Fragen?

Daniil Kryvoruchko · Gym Plan Generator · BFH Passerelle